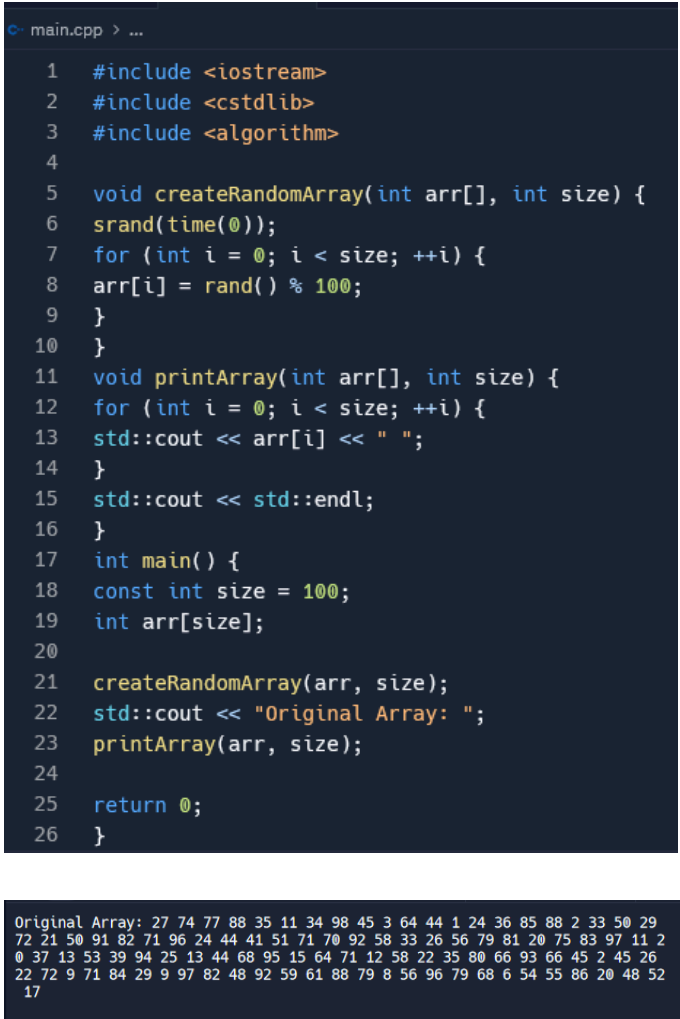


Activity No. 7	
SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: 10/16/24
Section: CPE21S1	Date Submitted: 10/16/24
Name(s): Katarina Nicole R. Delavin`	Instructor: Maria Rizette Sayo
6. Output	
Code + Console Screenshot	 <pre> 1 #include <iostream> 2 #include <cstdlib> 3 #include <algorithm> 4 5 void createRandomArray(int arr[], int size) { 6 srand(time(0)); 7 for (int i = 0; i < size; ++i) { 8 arr[i] = rand() % 100; 9 } 10 } 11 void printArray(int arr[], int size) { 12 for (int i = 0; i < size; ++i) { 13 std::cout << arr[i] << " "; 14 } 15 std::cout << std::endl; 16 } 17 int main() { 18 const int size = 100; 19 int arr[size]; 20 21 createRandomArray(arr, size); 22 std::cout << "Original Array: "; 23 printArray(arr, size); 24 25 return 0; 26 } </pre> Original Array: 27 74 77 88 35 11 34 98 45 3 64 44 1 24 36 85 88 2 33 50 29 72 21 50 91 82 71 96 24 44 41 51 71 70 92 58 33 26 56 79 81 20 75 83 97 11 2 0 37 13 53 39 94 25 13 44 68 95 15 64 71 12 58 22 35 80 66 93 66 45 2 45 26 22 72 9 71 84 29 9 97 82 48 92 59 61 88 79 8 56 96 79 68 6 54 55 86 20 48 52 17
Observations	The code focuses solely on generating and displaying a random array. It doesn't perform any further operations or analysis on the array.
Table 7-1. Array of Values for Sort Algorithm Testing	

Code + Console Screenshot

```
1  #include <iostream>
2  template <typename T>
3
4  void bubbleSort(T arr[], size_t arrSize) {
5      for ( int i = 0; i < arrSize; i++) {
6          for (int j = i + 1; j < arrSize; j++) {
7              if (arr[j] > arr[i]) {
8
9                  std::swap(arr[j], arr[i]);
10             }
11         }
12     }
13 }
14
15 int main() {
16     int arr[] = {13, 25, 17, 18, 30, 5, 24};
17     size_t arrSize = sizeof(arr) / sizeof(arr[0]);
18
19     std::cout << "Original array: ";
20     for (int i = 0; i < arrSize; i++) {
21         std::cout << arr[i] << " ";
22     }
23     std::cout << std::endl;
24
25     bubbleSort(arr, arrSize);
26
27     std::cout << "Sorted array: ";
28     for (int i = 0; i < arrSize; i++) {
29
30         std::cout << arr[i] << " ";
31     }
32     std::cout << std::endl;
33
34     return 0;
35 }
```

```
Original array: 13 25 17 18 30 5 24
Sorted array: 30 25 24 18 17 13 5
```

Observations

The bubble sort method iteratively goes over the list, contrasting similar items, and switching them if the sequence is incorrect.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
#include <iostream>

template <typename T>
int Routine_Smallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++){
        if (arr[j] < smallestElem){
            smallestElem
            = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], const int N){
    for (int i = 0; i < N - 1; i++){
        int POS = Routine_Smallest(arr, i, N);
        std::swap(arr[i], arr[POS]);
    }
}

int main(){
    int arr[]
    = {13, 25, 17, 18, 30, 5, 12};
    const int N = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original array: ";
    for (int i = 0; i < N; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    selectionSort(arr, N);

    std::cout << "Sorted array: ";
    for (int i = 0; i < N; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

```
Original array: 13 25 17 18 30 5 12
Sorted array: 5 12 13 17 18 25 30
```

Observations

Using a template function called selectionSort, which can handle arrays of any data type T, the code implements a selection sort algorithm. To determine which element in the array's unsorted section is the smallest, the method employs the help of the function Routine_Smallest.

Table 7-3. Selection Sort Algorithm

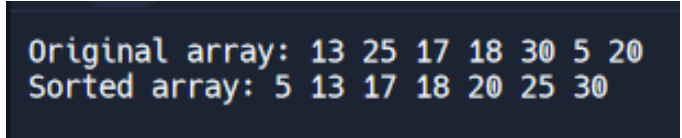
Code + Console Screenshot	<pre> 1 #include <iostream> 2 3 template <typename T> 4 void insertionSort(T arr[], const int N) { 5 for (int K = 1; K < N; K++) { 6 T temp = arr[K]; 7 int J = K - 1; 8 9 while (J >= 0 && arr[J] > temp) { 10 arr[J + 1] = arr[J]; 11 J--; 12 } 13 arr[J + 1] = temp; 14 } 15 } 16 17 int main() { 18 int arr[] = {13, 25, 17, 18, 30, 5, 20}; 19 const int N = sizeof(arr) / sizeof(arr[0]); 20 21 std::cout << "Original array: "; 22 for (int i = 0; i < N; i++) { 23 std::cout << arr[i] << " "; 24 } 25 std::cout << std::endl; 26 27 insertionSort(arr, N); 28 29 std::cout << "Sorted array: "; 30 for (int i = 0; i < N; i++) { 31 std::cout << arr[i] << " "; 32 } 33 std::cout << std::endl; 34 35 return 0; 36 } </pre> 
Observations	This code also uses selection sort just like in the table 7-3 but in insertion form to sort the elements from the original array from sorted array.

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

8. Conclusion

9. Assessment Rubric